VEHICLE RENTAL MANAGEMENT SYSTEM

RDBMS Lab Project Report

Student Name	Udith Praveen
Registration Number	2502052183
Branch / Section	B.tech / O4
Semester	3
Course	Relational Database Management Systems (RDBMS) Lab
Project Type	Database & Front-End Application Integration
Technologies	Python 3.x, MySQL, Terminal Interface
GitHub Link	github.com/Udith-Praveen/vehicle-rental-management-system

Project Summary: This project is a simple vehicle rental management system built using MySQL and Python. It manages car inventory, customer information, rental bookings, returns with billing calculation, and vehicle maintenance logs through an interactive terminal interface. The database uses Primary Keys and Foreign Keys to keep all records connected and accurate, and uses SQL JOIN queries to display human-readable rental details.

Section	Topic
Section 1	Introduction and Problem Statement
Section 2	Database Design and Schema
Section 3	SQL Queries and Operations
Section 4	Python Front-End Application
Section 5	Project Demonstration Walkthrough
Section 6	Project Contribution
Section 7	References

1. Introduction and Problem Statement

Car rental companies deal with daily operations like tracking available vehicles, registering customer licenses, recording rental dates, and calculating payments upon return. Managing this on paper or spreadsheets easily leads to mistakes such as double booking a vehicle or losing track of customer records.

The goal of this project is to create a clean, working database in MySQL connected to a simple Python terminal interface. It allows an operator to add vehicles, register customers, rent out cars, return cars with automatic billing, and view all records.

2. Database Design and Schema

The database consists of 5 tables. Each table has a Primary Key to identify rows uniquely, and Foreign Keys are used to connect related information between tables.

Table	Primary Key	Foreign Key	Columns & Purpose
Customer	`CustomerID`	None	FirstName, LastName, LicenseNumber (Unique), PhoneNumber. Stores customer details.
Vehicle	`VehicleID`	None	Make, Model, LicensePlate (Unique), DailyRate, Status. Stores vehicle inventory.
Rental	`RentalID`	CustomerID, VehicleID	RentalDate, ReturnDate, TotalCost. Links customer to rented car.
Payment	`PaymentID`	RentalID	PaymentDate, Amount, PaymentMethod. Records payment transactions.
Maintenance	`MaintenanceID`	VehicleID	ServiceDate, Description, Cost. Logs vehicle repairs and servicing.

Table Relationships (1 : M):

- **Customer to Rental (1 : M):** One customer can rent vehicles multiple times.
- **Vehicle to Rental (1 : M):** One vehicle can be rented repeatedly on different dates.
- **Rental to Payment (1 : M):** A rental can have one or more payments recorded.
- **Vehicle to Maintenance (1 : M):** A vehicle can have multiple service entries over time.

Referential Integrity:

Foreign Keys ensure that a rental cannot be created for a CustomerID or VehicleID that does not exist in the database. This prevents invalid or orphaned records.

3. SQL Queries and Operations

The project uses standard SQL commands to create the database structure, insert records, update status, and query data.

A. Table Creation (DDL)

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    LicenseNumber VARCHAR(30) UNIQUE NOT NULL,
    PhoneNumber VARCHAR(15) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE Vehicle (
    VehicleID INT PRIMARY KEY,
    Make VARCHAR(50) NOT NULL, Model VARCHAR(50) NOT NULL,
    LicensePlate VARCHAR(20) UNIQUE NOT NULL,
    DailyRate DECIMAL(8,2) NOT NULL, Status VARCHAR(20) DEFAULT 'Available'
) ENGINE=InnoDB;

CREATE TABLE Rental (
    RentalID INT PRIMARY KEY,
    CustomerID INT NOT NULL, VehicleID INT NOT NULL,
    RentalDate DATE NOT NULL, ReturnDate DATE,
    TotalCost DECIMAL(10,2) DEFAULT 0.00,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID) ON DELETE CASCADE,
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID) ON DELETE CASCADE
) ENGINE=InnoDB;
```

B. Inserting and Updating Records (DML)

```
-- Renting a vehicle (Insert rental record and change car status to 'Rented')
INSERT INTO Rental (RentalID, CustomerID, VehicleID, RentalDate, TotalCost)
VALUES (501, 1, 101, '2026-10-25', 0.00);
UPDATE Vehicle SET Status = 'Rented' WHERE VehicleID = 101;

-- Returning a vehicle (Set return date, total cost, and mark car 'Available')
UPDATE Rental SET ReturnDate = '2026-10-28', TotalCost = 135.00 WHERE RentalID = 501;
UPDATE Vehicle SET Status = 'Available' WHERE VehicleID = 101;
```

C. Querying Data with SQL INNER JOIN (DQL)

Instead of showing raw numeric IDs, an INNER JOIN combines rows from Rental, Customer, and Vehicle tables to display the customer's name, car make, and model together:

```
SELECT
    r.RentalID,
    CONCAT(c.FirstName, ' ', c.LastName) AS Customer,
    CONCAT(v.Make, ' ', v.Model) AS Vehicle,
    v.LicensePlate,
    r.RentalDate,
    IFNULL(r.ReturnDate, 'Pending') AS ReturnDate,
    r.TotalCost
FROM Rental r
JOIN Customer c ON r.CustomerID = c.CustomerID
JOIN Vehicle v ON r.VehicleID = v.VehicleID
ORDER BY r.RentalID DESC;
```

4. Python Front-End Application

The user interface is built in Python using mysql.connector. It provides a clear terminal menu where the user selects options to interact with the database. When an option is selected, Python executes the SQL query on MySQL and displays results neatly in ASCII tables.

```
------------------------------------------ | Formatted Query Output (Show Available Vehicles):
     VEHICLE RENTAL MANAGEMENT SYSTEM     | +------------+---------+---------+---------------+------------+-----------+
------------------------------------------ | | Vehicle ID | Make    | Model   | License Plate | Daily Rate | Status    |
 1. Add Vehicle        8. Rent a Vehicle   | +------------+---------+---------+---------------+------------+-----------+
 2. Show All Vehicles  9. Return a Vehicle | | 102        | Honda   | City    | KA-05-AB-5678 | 50.00      | Available |
 3. Available Vehicles 10. Show All Rentals| | 103        | Hyundai | Creta   | MH-02-CD-9999 | 65.00      | Available |
 4. Remove Vehicle     11. Log Payment     | +------------+---------+---------+---------------+------------+-----------+
 15. Setup DB Tables   0. Exit Application |
```

5. Project Demonstration Walkthrough

- **Database Setup:** Option 15 creates the database and all 5 tables with Primary and Foreign Keys.
- **Adding Records:** Options 1 and 5 insert new vehicles and register customer details.
- **Renting a Vehicle:** Option 8 records the rental and automatically updates the vehicle status to 'Rented'.
- **Viewing Rentals:** Option 10 runs the INNER JOIN query to show active and completed rentals with full customer and car details.
- **Returning a Vehicle:** Option 9 calculates the total rental fee, records the return date, and sets the car status back to 'Available'.

6. Project Contribution

Student Name	Registration Number	Contribution
Udith Praveen	**2502052183**	• Designed the relational database schema, tables, and key constraints. • Wrote the complete SQL script in schema.sql. • Implemented database connection handling in db_config.py. • Built the terminal menu, ASCII table formatting, and CRUD functions in main.py. • Implemented the multi-table INNER JOIN query and prepared the documentation.

7. References

1. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
2. A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
3. MySQL 8.0 Reference Manual, Oracle Corporation. URL: https://dev.mysql.com/doc/refman/8.0/en/
4. Python MySQL Connector Documentation. URL: https://dev.mysql.com/doc/connector-python/en/